

HTTPS (Linux ou Windows)

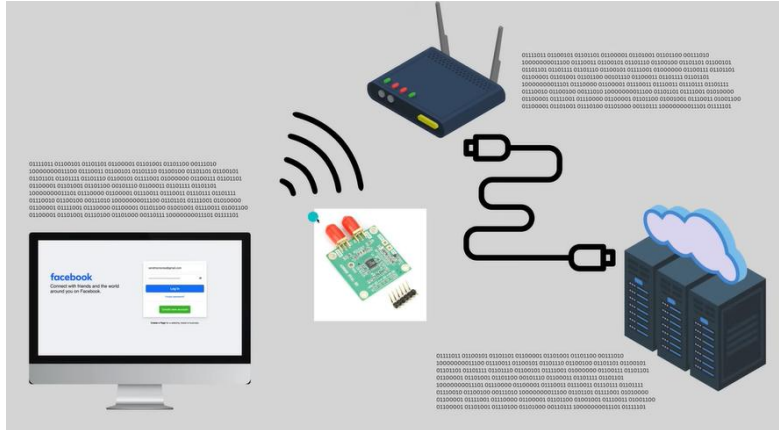
1. Debate do funcionamento do HTTPS

- Definição do problema (envio de chave simétrica)
- Papel da chave assimétrica (pública e privada) – para assinatura e proteção.
- Papel do órgão certificador

Vídeo bem básico e didático em <https://www.youtube.com/watch?v=EnY6fSng3Ew>

Vídeo mais detalhado em <https://www.youtube.com/watch?v=ZkL10eoG1PY>

Problema inicial: enviar de forma segura informações sensíveis para um servidor.



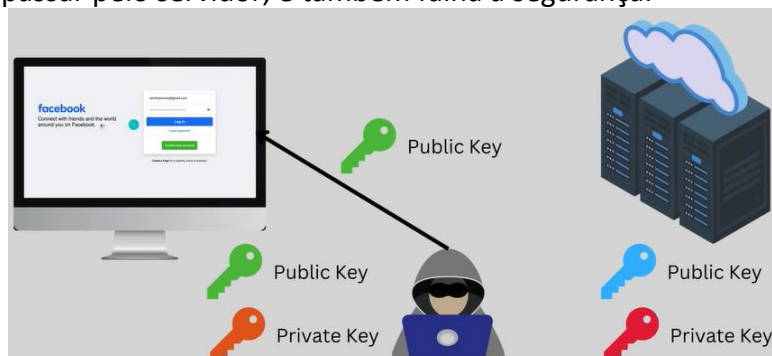
HTTP, sem segurança... Pode ser interceptado (wireshark, por exemplo).

Alternativa 1: usar chave simétrica. Problema: como passar a chave simétrica de forma segura ao servidor? Se o método não for confiável, atacante pode saber essa chave simétrica também, e falha a segurança.



Enviar chave simétrica junto... Pode ser interceptado (wireshark, por exemplo).

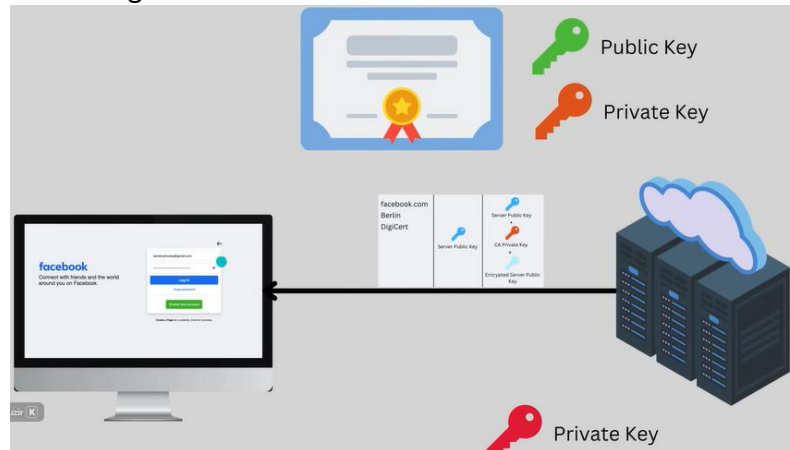
Alternativa 2: usar chave assimétrica. Servidor envia sua chave **pública** ao cliente, que a usa para criptografar chave simétrica. Só pode ser decriptografada pela chave **privada** do servidor. Problema: atacante pode se fazer passar pelo servidor, e também falha a segurança.



Usar chave pública do servidor... Falha com ataque "man in the middle"

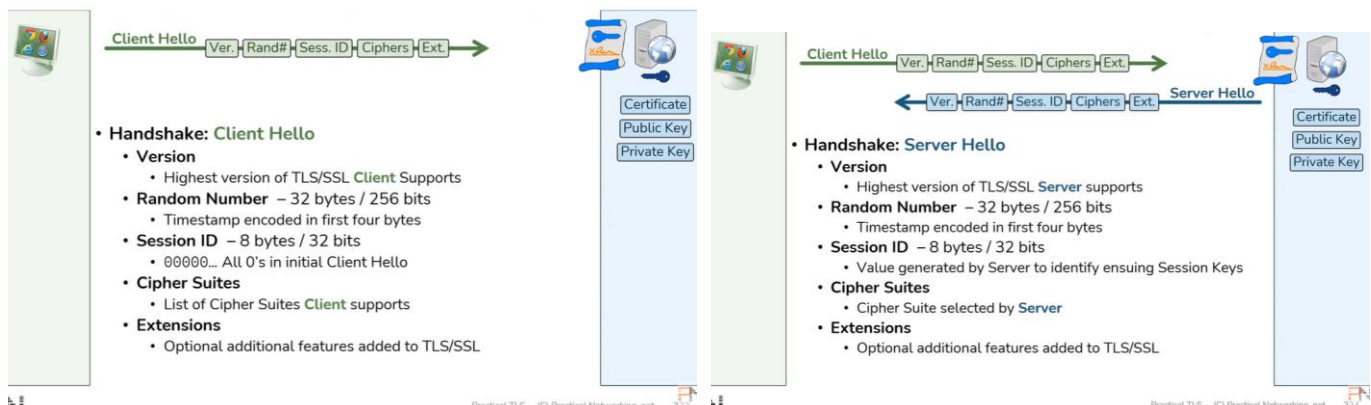
Alternativa 3: Servidor envia sua **chave pública** ao cliente, porém antes faz os seguintes passos:

- Solicita para autoridade certificadora (CA) criptografar certificado (que contém a chave pública) do servidor com chave privada da autoridade certificadora (autoridade certificadora assina o certificado)
- Criptografa esse certificado assinado pela CA com sua chave privada e envia para cliente.
- Cliente deve usar a chave pública do servidor MAIS a chave pública da autoridade certificadora para obter a chave pública do servidor com maior garantia.
- Agora sim, o cliente pode utilizar a **chave pública do servidor para criptografar a chave simétrica** com maior garantia.

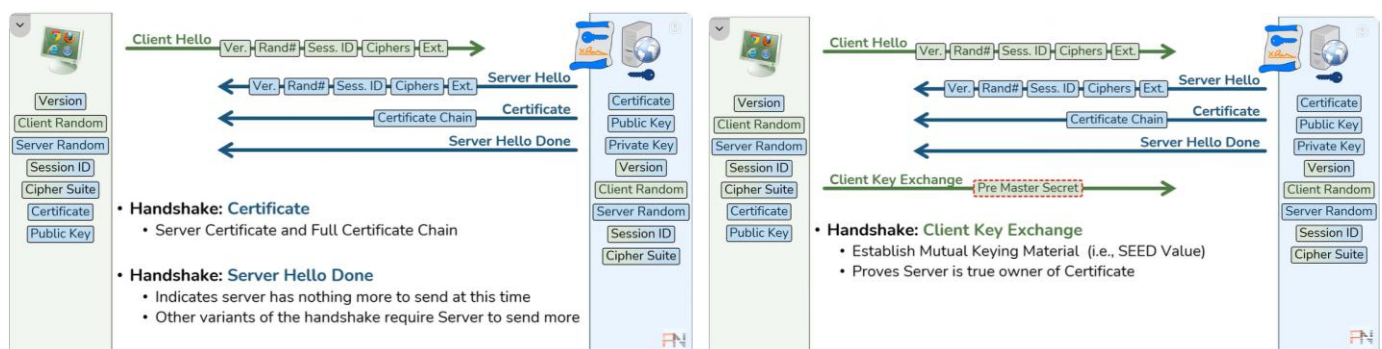


Usar certificado... ou cadeia de certificados

Detalhamento por pacotes de rede (referência... não tem tempo de analisar a fundo. Ver vídeo)



Obs: timestamp do “Random Number” com precisão de um segundo. Não permite dois RN iguais. RN é usado para gerar “master secret” no futuro.



Obs: Certificado enviado pelo servidor encriptado com chave privada do servidor e chave privada da agência certificadora. Decriptado pelo cliente com chave pública da agência certificadora e chave pública do servidor.

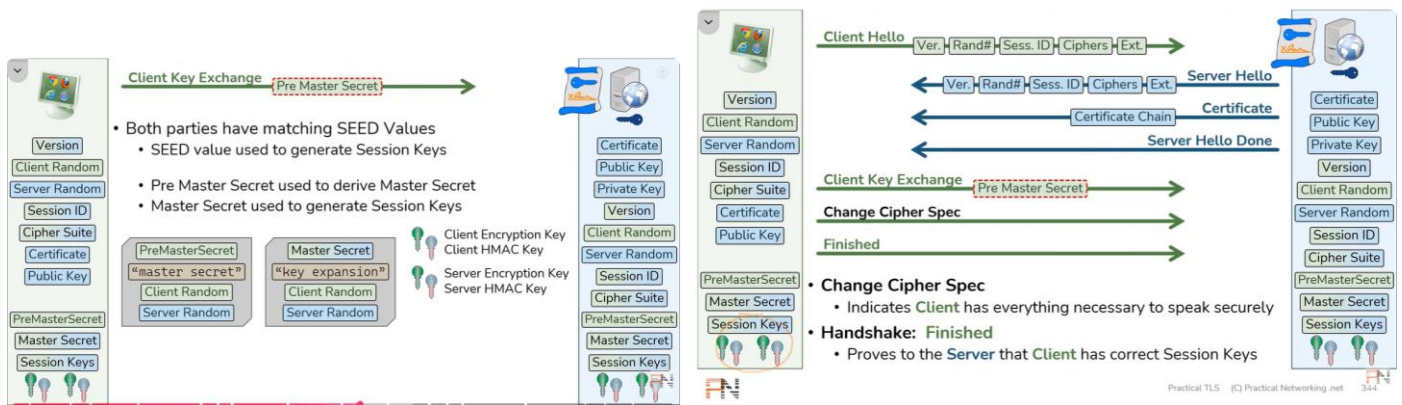


Fig1: caixa da esquerda gera "Master Secret", e da direita gera "Session Key". São geradas chaves simétricas com as "Session Keys". Cliente e servidor vão possuir chaves simétricas diferentes, criando dois túneis, cada um criptografando dados de um lado para outro.

Fig2: as mensagens de "change cipher spec" são para confirmar que servidor e cliente calcularam corretamente as chaves simétricas (ver fig1).

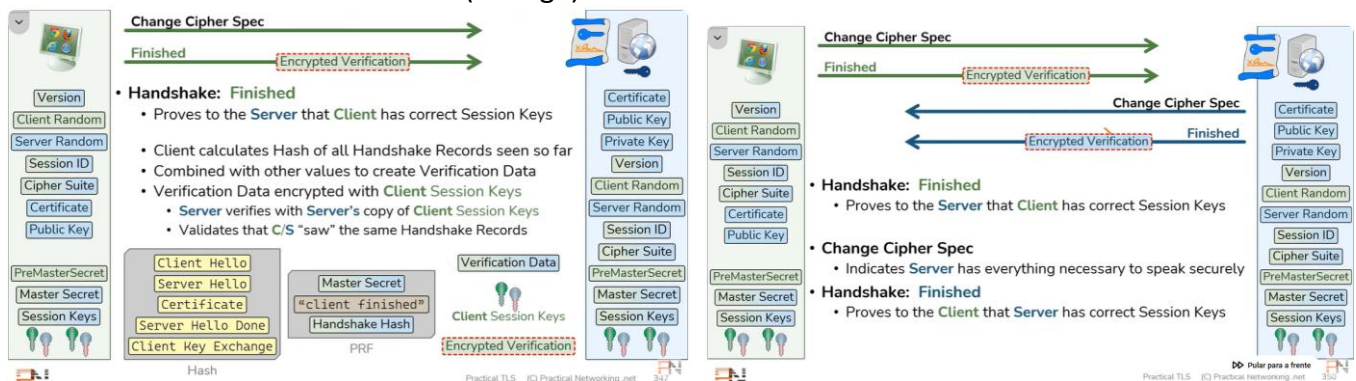


Fig1 e Fig2: Na mensagem de "Finished" são enviados *hashes* de todos *handshakes* (mais outros valores), e isso é criptografado com a chave simétrica do cliente. Essa é a mensagem de "Encrypted Verification".

2. Escolha um site seguro (por exemplo, <https://ufrgs.br>). Verifique o certificado utilizado. Para isso, no chrome, clicar nos 3 pontos verticais + "mais ferramentas" + "ferramentas de desenvolvedor" e clicar na aba "segurança".
 - a) Qual a versão de TLS utilizada?
 - b) Qual o órgão certificador?
 - c) Qual a validade do certificado?
3. Abra o arquivo "_Lab2_tls-handshake-greer.pcap"
 - a) Mudar a cor de fundo dos pacotes TLS: DICA: ir no pacote 4, em "Transport Layer Security" e abaixo de TLSv1.3. Achar onde diz "content type: Handshake (22)". Arrastar com o mouse para o filtro de exibição do wireshark. Copiar o que aparece lá (tls.record.content_type==22) e adicionar como uma nova regra de cores.
 - b) Qual a versão mais baixa, mais alta e "real" de TLS negociada?
 - c) Qual a versão de TLS negociada pelo servidor?
 - d) O campo "session ID" é igual para o cliente e servidor?
 - e) Quantos algoritmos possíveis de troca de chaves possui o cliente (pacote 4 - "cipher suite") e qual foi negociado na conexão (pacote 6)?
 - f) Qual o nome do servidor consultado? Após descobrir o nome, faça um filtro de exibição com: *tcp contains "nnn"* ou *frame contains "nnn"*.
 - g) O restante está criptografado. Para descriptografar, vamos utilizar o sslkeylog disponibilizado no moodle. Para isso, no wireshark, vá em "Editar" + "Preferências" + "Protocolos". Escolha o TLS. Onde diz "(Pre)-Master-Secret log filename", coloque o caminho do arquivo fornecido (pode usar "procurar" e clicar em cima).
 - h) Observa-se agora que aparecem os HTTPs que antes estavam criptografados, bem como algumas mensagens de handshake também criptografadas. Observe os HTTPs.
 - i) Observe o pacote 57, onde o wireshark "remontou" vários pacotes da captura. Veja as várias abas. Observe a aba "uncompressed entity body" para ver o HTML aberto.

4. Veja a figura a seguir. Tente descobrir o motivo provável do servidor ter recusado a conexão – pacote 7

Time	Delta	Source	Destination	Protocol	Total Len	TCP Segment	Info
1 01:0...	0.000000	192.168.0.1	10.10.10.1	TCP	52	0 49923 → https(443) [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=25	
2 01:0...	0.001127	10.10.10.1	192.168.0.1	TCP	52	0 https(443) → 49923 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=	
3 01:0...	0.000129	192.168.0.1	10.10.10.1	TCP	40	0 49923 → https(443) [ACK] Seq=1 Ack=1 Win=262144 Len=0	
4 01:0...	0.000000	192.168.0.1	10.10.10.1	TLSv1	144	104 Client Hello	
5 01:0...	0.000995	10.10.10.1	192.168.0.1	TCP	40	0 https(443) → 49923 [ACK] Seq=1 Ack=105 Win=7680 Len=0	
6 01:0...	0.000000	10.10.10.1	192.168.0.1	TCP	40	0 [TCP Window Update] https(443) → 49923 [ACK] Seq=1 Ack=105 Wi	
7 01:0...	0.000000	10.10.10.1	192.168.0.1	TLSv1	47	7 Alert (Level: Fatal, Description: Protocol Version)	
8 01:0...	0.000650	10.10.10.1	192.168.0.1	TCP	40	0 https(443) → 49923 [FIN, ACK] Seq=8 Ack=105 Win=6144 Len=0	
9 01:0...	0.000032	192.168.0.1	10.10.10.1	TCP	40	0 49923 → https(443) [ACK] Seq=105 Ack=8 Win=261888 Len=0	
10 01:0...	0.000000	192.168.0.1	10.10.10.1	TCP	40	0 49923 → https(443) [ACK] Seq=105 Ack=9 Win=261888 Len=0	
11 01:0...	0.000000	192.168.0.1	10.10.10.1	TCP	40	0 49923 → https(443) [FIN, ACK] Seq=105 Ack=9 Win=261888 Len=0	
12 01:0...	0.000502	10.10.10.1	192.168.0.1	TCP	40	0 https(443) → 49923 [ACK] Seq=9 Ack=106 Win=6144 Len=0	

5. Capture e analise um fluxo HTTPS próprio

a) Configure a variável de ambiente (ou de sessão):

- No Windows:** clicar em “Iniciar” e pesquisar por “editar variáveis de ambiente para sua conta”. Embaixo clicar em “Variáveis de Ambiente”. Nas **variáveis de usuário**, criar uma nova (Novo). No campo “Nome da Variável”, usar “SSLKEYLOGFILE” e no “Valor da Variável” usar “local\nomearquivo.log”, como, por exemplo, “C:\Users\Valter Roesler\Desktop\sslkeylog.log”. Feche o navegador (reiniciar o navegador).
- No Linux:** abrir um novo terminal (CTRL+ALT+T). Digitar mantendo as aspas: `> export SSLKEYLOGFILE = "/home/$USER/sslkey.log"`. Após isso, o navegador DEVE ser aberto pelo terminal. Para Firefox, utilizar `> firefox &`. Para chromium, digitar `> chromium-browser &`

- No Wireshark, vá em “Editar” + “Preferências” + “Protocolos”. Escolha o TLS. Onde diz “(Pre)-Master-Secret log filename”, coloque o caminho do arquivo criado (pode usar “procurar” e clicar em cima).
- Capture uma consulta no navegador para www.ufrgs.br. Naturalmente vai usar HTTPS.
- Localize o fluxo desejado. Uma forma é pesquisar no filtro de exibição algo tipo: frame contains “ufrgs”. Localizando um pacote TLS ou TCP, dá pra fazer clique da direita e ir em “seguir + fluxo TCP”. Agora você deve ter a captura filtrada pelo fluxo TCP / TLS.
- Localize os pacotes: a) Client Hello; Server Hello; Certificate; Server Key Exchange; Server Hello Done; Client Key Exchange; HTTP.
- Qual a versão de TLS estabelecida?
- Qual o algoritmo de criptografia escolhido pelo servidor?
- Nas mensagens de “Hello”, observe também os campos de “Random” e “Session ID”. São importantes, apesar de não serem trabalhados em aula.